

Lab 04 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde stramien: het antwoord zelf, het mechanisme erachter, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — `COPY ../gemeenschappelijk/...`

Antwoord. De build ziet alleen de **context** — de map die je als argument meegeeft (`.`, hier dus `~/projecten/api/`). `../gemeenschappelijk` ligt daarbuiten. Buildah trekt het pad terug binnen de context (`possible escaping context directory`), vindt daar niets, en de build mislukt. `-f` wijst alleen de Dockerfile aan, niet de grens; een absoluut pad wordt net zo goed binnen de context getrokken; en `sudo` helpt niet, want dit is een grensprobleem, geen rechtenprobleem. De oplossing: bouwen vanuit de bovenliggende map (`podman build -f api/Dockerfile -t api:1.0 ~/projecten`) met paden als `COPY api/... gemeenschappelijk/...`, of `config.yml` vóór de build naar het project kopiëren — of nog beter: het bestand helemaal uit de image houden en het pas bij de uitvoering injecteren (lab 08).

Waarom. De context is een grens voor veiligheid en reproduceerbaarheid: een Dockerfile mag alleen afhangen van wat je hem expliciet geeft. Docker dwingt dat fysiek af (de context wordt gearchiveerd en naar de daemon gestuurd); bij Podman is het een regel die Buildah toepast — het resultaat is hetzelfde.

Nuance. Podman ondersteunt meerdere benoemde contexten: `podman build --build-context gemeenschappelijk=../gemeenschappelijk .` gevolgd door `COPY --from=gemeenschappelijk config.yml /app/`. Dat is de nette oplossing wanneer een gedeeld bestand echt in meerdere images thuishoort.

Voorbeeld.

```
podman build -f Dockerfile.buiten-context -t poging .  
# Error: building at STEP "COPY ../gemeenschappelijk/secret.txt /": ... possible escaping context  
directory error
```

Vraag 2 — `transferring context`, en dan niets meer

Antwoord. De client van Docker verpakt de **hele** map (1,1 GB) en stuurt ze naar de daemon nog vóór de eerste instructie draait: dat is de regel `transferring context`. Buildah van Podman leest de map ter plaatse — geen archief, geen overdracht — dus de traagheid verdwijnt. Het tweede risico blijft echter onverminderd bestaan: een `COPY . .` stopt `node_modules` en `.git` nog altijd **in de image** — 1,1 GB ballast, plus de volledige Git-geschiedenis (met eventuele geheimen erin) voor iedereen die de image downloadt. `.dockerignore` blijft dus verplicht; het levert geen snelheid meer op, het bewaakt de inhoud.

Waarom. De context speelt twee rollen: wat *verstuurd* wordt (alleen bij Docker) en wat *kopieerbaar* is. Podman schrapt de eerste kost, niet de tweede.

Nuance. `.git` in een image is een veelvoorkomend en ernstig lek: de geschiedenis bevat vaak credentials die "later" verwijderd zijn. En zonder `.dockerignore` doet een gewijzigd bestand in `node_modules` ook de cache van de `COPY` vervallen.

Voorbeeld.

```
podman build -q -f Dockerfile.alles -t t .          # image van 218 MB met node_modules
printf 'node_modules\n.git\n' > .dockerignore
podman build -q -f Dockerfile.alles -t t2 .        # 8,7 MB
```

Vraag 3 — `EXPOSE` en de poort die niet antwoordt

Antwoord. Nee, de image is in orde. `EXPOSE` is een **verklaring**: het documenteert dat de applicatie op 8080 luistert, en het voedt `podman ps` en `-P`. Het maakt geen enkele doorsturing vanaf de host aan. Zonder `-p 8080:8080` is de poort alleen bereikbaar vanuit het containerennetwerk.

Waarom. Een poort publiceren is een uitrolbeslissing (welke hostpoort, welke interface), geen eigenschap van de image. De image zegt "ik luister op 8080"; de beheerder beslist "ik maak ze bereikbaar op 18080".

Nuance. `-P` (hoofdletter) publiceert automatisch alle `EXPOSE`-poorten op willekeurige hostpoorten — daar bewijst de verklaring haar nut. En in rootless-modus mislukt `-p 80:8080` (geprivilegieerde poort); kies een poort ≥ 1024 of stel `net.ipv4.ip_unprivileged_port_start` in.

Voorbeeld.

```
podman run -d --name a mijn-api:1.0 && curl -m 2 localhost:8080      # mislukt
podman run -d --name b -p 8080:8080 mijn-api:1.0 && curl localhost:8080/actuator/health #
{"status":"UP"}
podman port b                                                         # 8080/tcp -> 0.0.0.0:8080
```

Vraag 4 — `RUN java` tegenover `CMD java`

Antwoord. A start de API **tijdens de build**: `RUN` voert zijn commando uit op het moment van bouwen, dus de API start, eindigt nooit... en de build blijft hangen (of, als de API toch stopt, houdt de image er alleen een nutteloze laag aan over). B is correct: `CMD` legt het commando vast dat `podman run` zal starten.

Waarom. `RUN` bereidt het bestandssysteem voor (installeren, compileren, kopiëren); `CMD` / `ENTRYPOINT` beschrijven het hoofdproces van de toekomstige container. Wie de twee door elkaar haalt, haalt bouwen en uitvoeren door elkaar.

Nuance. B zou nog beter zijn met `ENTRYPOINT` + `CMD` (vraag 5) en een `USER`. En `RUN java -jar` heeft wel degelijk een legitiem gebruik: een **eindige taak** uitvoeren tijdens de build, zoals `java -Djarmode=tools -jar app.jar extract` (lab 05).

Voorbeeld.

```
podman build -f A -t a .          # STEP 3/3: RUN java -jar ... - komt nooit tot een einde
podman build -f B -t b . && podman run -d -p 18080:8080 b
```

Vraag 5 — `ENTRYPOINT` + `CMD` tegenover `CMD` alleen

Antwoord. A: `podman run img` → `java -jar /app/api.jar --spring.profiles.active=prod`; `podman run img --debug` → `java -jar /app/api.jar --debug` (de `CMD` wordt vervangen, de `ENTRYPOINT` blijft). B: `podman run img` → hetzelfde volledige commando; `podman run img --debug` → probeert `-debug` **op zichzelf** uit te voeren, zonder `java`, en mislukt met `executable file not found`.

Alleen bij B werkt `podman run img sh` nog — de hele `CMD` wordt door `sh` vervangen. Bij A start `podman run img sh` gewoon `java -jar api.jar sh`; daar heb je `podman run --entrypoint sh img` nodig.

Waarom. Argumenten van `run` vervangen de `CMD` en komen achter de `ENTRYPOINT`. A past bij een "applicatie"-image, B bij een "tool"-image.

Nuance. In `exec`-vorm maken zowel A als B `java` PID 1. Een gangbare variant in bedrijven: `ENTRYPOINT ["java","-jar","app.jar"]` zonder `CMD`, en alle configuratie via omgevingsvariabelen — argumenten dienen dan alleen nog om te debuggen.

Voorbeeld.

```
timeout 5 podman run --rm api-lab:1.0 --debug | head -1      # Arguments recus : --debug
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo ok'     # ok
```

Vraag 6 — Tien seconden, `resorting to SIGKILL`, geen hooks

Antwoord. `CMD java -jar /app/api.jar` is een **shell**-vorm: de engine voert `/bin/sh -c "java -jar /app/api.jar"` uit. Op een Debian/Ubuntu-basis is `/bin/sh` `dash`, dat Java als kindproces start en zelf PID 1 blijft. `podman stop` stuurt `SIGTERM` naar PID 1 — de shell — en die geeft het niet door. Java krijgt het signaal nooit en zijn *shutdown hooks* draaien niet; na 10 seconden meldt Podman `resorting to SIGKILL` en maakt alles af (137). De correctie: `CMD ["java","-jar","/app/api.jar"]`. Op Alpine is `/bin/sh` de `ash` van busybox, die bij een eenvoudig commando **zichzelf vervangt**: Java wordt PID 1, ontvangt `SIGTERM`, en de bug blijft in de test onzichtbaar.

Waarom. Een POSIX-shell is niet verplicht om signalen door te geven aan zijn kinderen, en `dash` doet het niet. De `exec`-vorm haalt de shell weg, en daarmee het probleem.

Nuance. Zelfs Alpine redt een shell-`CMD` met `&&`, `|` of een variabele niet: dan moet de shell wél blijven. De regel "altijd `exec`-vorm" bespaart je het uit het hoofd leren van het gedrag van elke shell.

Voorbeeld.

```
podman exec s-deb ps -o pid,args | head -3      # 1 /bin/sh -c java ... 2 java ...
time podman stop s-deb                         # resorting to SIGKILL, 10 s, code 137
```

Vraag 7 — `$JAVA_OPTS` niet geïnterpreteerd

Antwoord. In `exec`-vorm is er **geen shell**, dus `$JAVA_OPTS` komt onaangeroerd bij Java aan, als een letterlijke tekenreeks van zes tekens. Twee correcties. (1) Expliciete shell-vorm met `exec` — `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]`; de prijs: een afhankelijkheid van `sh` en een minder leesbare regel. (2) De variabele schrappen — Java leest zelf `JAVA_TOOL_OPTIONS` uit de omgeving, dus `ENV JAVA_TOOL_OPTIONS="-Xmx512m"` met `ENTRYPOINT ["java","-jar","/app/api.jar"]`; de prijs: een melding `Picked up JAVA_TOOL_OPTIONS` op `stderr` bij het opstarten, en een variabele die op *alle* Java-processen in de container inwerkt.

Waarom. Variabelen uitbreiden is een dienst van de shell. De `exec`-vorm is een rij argumenten die rechtstreeks naar de systeemaanroep `execve` gaat.

Nuance. Correctie (1) zonder `exec` zou het probleem van vraag 6 opnieuw binnenhalen. En specifiek voor geheugen is `-XX:MaxRAMPercentage=75` beter dan een vaste `-Xmx`: de JVM past zich

aan de cgroup aan (lab 10).

Voorbeeld.

```
podman run --rm -e JAVA_TOOL_OPTIONS="-Xmx256m" api-lab:1.0 --debug 2>&1 | head -1  
# Picked up JAVA_TOOL_OPTIONS: -Xmx256m
```

Vraag 8 — `--build-arg` en het geheim

Antwoord. Nee. De waarde van een `ARG` staat niet in `Config.Env`, maar ze wordt wél vastgelegd in de **geschiedenis** van elke instructie die ze gebruikt, en in de buildcache. `podman history --no-trunc api:1.0` toont ze in klare tekst (`| 1 DB_PASSWORD=Secr3t! /bin/sh -c ...`). Wie de image heeft, heeft het wachtwoord.

Waarom. De geschiedenis legt vast hoe elke laag tot stand kwam, argumenten inbegrepen — precies dat maakt de cache mogelijk. Een `ARG` is invoer voor de build, dus voor de cache, dus voor de geschiedenis.

Nuance. De goede praktijk: een geheim heeft bij de *build* niets te zoeken. Is het toch onvermijdelijk (een private Maven-repository, bijvoorbeeld), dan stelt `RUN --mount=type=secret,id=settings ...` het beschikbaar tijdens één instructie zonder het ooit in een laag te schrijven (lab 08). Een multi-stage-build beschermt alleen als het geheim uitsluitend in een weggegooid stage gebruikt wordt (lab 05).

Voorbeeld.

```
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | grep DB_PASSWORD  
# | 1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build met $DB_PASSWORD" > /trace.txt
```

Vraag 9 — Zes minuten per Maven-build

Antwoord.

```
WORKDIR /app  
COPY pom.xml .  
RUN mvn -q dependency:go-offline      # download van de afhankelijkheden, gecachet  
COPY src ./src  
RUN mvn -q package -DskipTests      # alleen compilatie
```

De cache hergebruikt een instructie wanneer haar tekst **en** haar invoer ongewijzigd zijn. `pom.xml` verandert zelden, dus de laag `dependency:go-offline` — de vijf minuten downloadwerk — blijft op `Using cache` staan. Verandert er een `.java`-bestand, dan wordt alleen de compilatie opnieuw gedaan. De build blijft traag telkens wanneer `pom.xml` verandert — een afhankelijkheid erbij of een versie omhoog — want dan vervalt de laag met de afhankelijkheden.

Waarom. `COPY . /app` zet *alle* code vóór Maven; elke commit doet de kopie vervallen, en daarmee alles wat erna komt.

Nuance. `dependency:go-offline` is niet perfect (sommige plugins downloaden alsnog tijdens `package`). Een `RUN --mount=type=cache,target=/root/.m2` (lab 05) bewaart de lokale repository tussen builds door, zelfs wanneer `pom.xml` verandert. En bij Podman heeft geen van die winsten een daemon nodig: de cache zit in je gebruikersopslag.

Voorbeeld.

```
time podman build -t api-lab:2.1 .      # RUN ... sleep 5: --> Using cache; 0,7 s
```

Vraag 10 — Drie apt- RUN s

Antwoord. (1) **Drie lagen in plaats van één:** de apt-lijsten (`/var/lib/apt/lists` , ~40 MB) komen in laag 2 terecht; laag 3 verbergt ze alleen maar, dus de image blijft de 40 MB meesleuren. (2) **Een geïsoleerde** `apt-get update` wordt gecachet: weken later hergebruikt een wijziging aan de `install`-regel verouderde indexen (pakketten niet gevonden, oude versies). (3) **Geen** `--no-install-recommends` , en `vim` in een productie-image: tientallen MB overbodige pakketten, en evenveel extra aanvalsoppervlak. De correcte versie:

```
RUN apt-get update \  
&& apt-get install -y --no-install-recommends curl \  
&& rm -rf /var/lib/apt/lists/*
```

Waarom. Een laag is onveranderlijk; opruimen werkt alleen in de laag die de bestanden heeft aangemaakt. En de cache werkt instructie per instructie: `update` en `install` horen bij elkaar.

Nuance. Op Alpine doet `apk add --no-cache curl` dit alles in één regel. En `vim` in een image valt nooit te verantwoorden: gebruik `podman exec` met een tijdelijke editor, of helemaal geen editor (distroless image, lab 05).

Voorbeeld.

```
podman history img --format 'table {{.Size}}\t{{.CreatedBy}}'  # de laag "rm -rf" is 0B, die  
erboven 40 MB
```

Vraag 11 — Using cache en dan alles herbouwd

Antwoord. De regel: **een vervallen instructie doet alle volgende vervallen**, en de cache wordt van boven naar onder afgelopen. Dag 1: alleen de laatste `COPY` is veranderd en alles erboven is identiek → acht keer `Using cache` , daarna worden de laatste twee stappen herbouwd. Dag 2: een instructie invoegen op de derde positie verandert de Dockerfile vanaf regel 3 → stappen 3 tot 10 zijn nieuw voor de cache en worden herbouwd — ook al is hun inhoud niet veranderd.

Waarom. De cachesleutel van een stap is (ouderlaag, instructie, invoer). Verander de ouderlaag en je verandert de sleutel van alles wat eronder staat.

Nuance. Daarom horen veranderlijke `ENV` , `ARG` en `LABEL` (buildnummer, datum) **achteraan** in een Dockerfile, en stabiele metadata vooraan. Een `ARG BUILD_DATE` op regel 2 doet alles vervallen, bij elke build opnieuw.

Voorbeeld.

```
podman build -t api-lab:3.1 .      # STEP 3/5: COPY ... (opnieuw) en dan STEP 4/5: RUN ... sleep 5  
(ook opnieuw)
```

Vraag 12 — `ADD` tegenover `COPY`

Antwoord. Twee gedragingen die eigen zijn aan `ADD`: het **pakt** een lokaal archief (`.tar`, `.tar.gz`, `.tar.xz`) automatisch uit naar de bestemming, en het **downloadt** URL's. De officiële aanbeveling is `COPY`, omdat beide gedragingen impliciet zijn: een `ADD bestand.tar.gz /app/` pakt uit terwijl je het archief wilde kopiëren, en een gedownloade URL komt zonder verificatie, zonder cache, en zonder mogelijkheid tot `rm` in dezelfde laag. Het enige te verantwoorden geval: een **lokaal** archief uitpakken in één instructie (`ADD rootfs.tar.gz /`).

Waarom. Een Dockerfile moet zonder verrassingen te lezen zijn, en `COPY` doet precies één ding. Voor een URL is `RUN curl ... && tar ... && rm ...` in één enkele `RUN` expliciet én op te ruimen.

Nuance. Beide instructies aanvaarden `--chown` (en `--chmod`), handig vóór een `USER`. En `COPY --from=` (lab 05) heeft geen `ADD`-equivalent.

Voorbeeld.

```
ADD app.tar.gz /opt/          # /opt/app/... uitgepakt
COPY app.tar.gz /opt/         # /opt/app.tar.gz als zodanig
```

Vraag 13 — `USER` te vroeg, en `HUSER 100999`

Antwoord. `USER` geldt voor elke instructie die erna komt, `RUN` inbegrepen. Meteen na `FROM` geplaatst laat het `apt-get install` en `mkdir` draaien als UID 1000, en die mag niet schrijven in `/usr`, `/var` of `/`: vandaar `Permission denied`. In een goed geschreven Dockerfile staat `USER` **net vóór** `ENTRYPOINT / CMD`, nadat alles geïnstalleerd is, de mappen aangemaakt zijn en de eigenaar goed staat (`chown`, `COPY --chown`). In rootless-modus wordt UID 1000 van de container via `/etc/subuid` op de host afgebeeld: de eerste "extra" UID (1) komt overeen met 100000, dus `1000 → 100999`. `USER` verdient nog altijd zijn plaats: (a) het ontnemt de applicatie de root-rechten *binnen* de container (imagebestanden wijzigen, luisteren op poort 80, pakketten installeren); (b) dezelfde image zal ook onder Docker of Kubernetes draaien, waar root echt root is; (c) beveiligingsscaners en toelatingsbeleid weigeren images zonder `USER`.

Waarom. De `user`-namespace beschermt de *host*; `USER` beschermt de *container* en zijn inhoud. Beide lagen vullen elkaar aan.

Nuance. `USER 1000:1000` werkt ook zonder de gebruiker aan te maken (Podman voegt zelfs on the fly een `/etc/passwd`-regel toe), maar sommige programma's verwachten een `HOME` of een naam: `RUN adduser -D -u 1000 app` gevolgd door `USER app` is robuuster.

Voorbeeld.

```
podman top u user,huser          # 1000 100999
podman run --rm --entrypoint sh api-lab:user-ok -c 'touch /app/x'      # Permission denied: goed zo.
```

Vraag 14 — "Eén enkele `RUN`"

Antwoord. De collega heeft gelijk wanneer bestanden die de ene instructie aanmaakt, door een andere verwijderd worden (installeren + opruimen, uitpakken + archief wissen): verdeeld over aparte lagen blijven die bestanden in de image zitten. Hij heeft ongelijk zodra de groepering stabiel en vluchtig vermengt: één `RUN` die de afhankelijkheden downloadt **én** de code compileert, vervalt bij elke commit en downloadt dus telkens alles opnieuw; en één laag van 300 MB gaat bij

elke `push` volledig opnieuw over de lijn, terwijl vijf lagen — waarvan vier stabiel — alleen het verschil versturen.

Waarom. Het aantal lagen kost op zich bijna niets. Wat telt, is **welke bestanden in welke laag zitten** (grootte) en **hoe vaak elke laag verandert** (cache en overdracht).

Nuance. Praktische vuistregel: één `RUN` per "eenheid van verandering" — systeempakketten (stabiel), afhankelijkheden (semi-stabiel), code (vluchtig). Multi-stage-builds (lab 05) en de laagindeling van de Spring Boot-JAR passen precies die logica toe.

Voorbeeld.

```
podman history mijn-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # één laag per eenheid  
van verandering
```